

MEAM 620 Project 3 Report with Extra Credit

Parra Wang

1 Overview

The goal of this project was to combine my previous planning, control, and VIO code into one full quadrotor autonomy stack. In Project 1, the simulator used the true state for feedback, so the controller could follow fairly aggressive trajectories. In Project 3, the controller uses the estimated state from VIO. This made the problem harder because estimation error and tracking error both matter. A path that looked safe in the planner could still collide if the trajectory was too fast, too close to obstacles, or too hard for the quadrotor to track.

My final main-task approach uses A* on a 3D occupancy grid, then converts the grid path into a smaller set of waypoints. I first used the reduced-waypoint method from Project 1.4, but it was still not enough for the switchback map. After trying to fix that with only the waypoint shortcutting, I remembered that the RDP idea from my Project 1.3 submission was already working pretty well, even if it was not the highest score by itself. So I combined those two ideas. In the final version, I used a smooth polynomial trajectory for most maps and an RDP simplified trajectory for the window and switchback style maps. The final PID-style gain tuning also helped the quadrotor track the aggressive switchback trajectory without colliding.

For the extra credit, I extended the planner from a one-shot global planner into a local replanning planner with limited sensor range. The quadrotor updates a local occupancy map around its current position, chooses a short local goal, replans a short segment, and repeats this process until the global goal is reached. This made the planner slower than the vanilla global version, but it also made the autonomy stack closer to the unknown-environment setting described in the project handout.

2 Main Task Code Changes

The main changes from Projects 1 and 2 were in `world_traj.py` and `se3_control.py`. I mostly kept the Project 2 VIO implementation and focused on making the planner and controller work better when VIO was in the loop.

In the planner, I changed the margin and speed based on the map. For shorter maps like maze and over-under, I used a moderate margin and a faster smooth trajectory. The reduced-waypoint idea helped a lot, but on its own it still left the switchback map too close to failure. That was the part where I brought back RDP from Project 1.3 and combined it with the newer waypoint reduction method. For more direct routes like the window map, I used the RDP version with a larger planning margin and a faster straight-line segment trajectory. For long maps like switchback, I also used the RDP trajectory because it followed the A* route more reliably than the smooth polynomial version. I also added checks for cases where graph search returns no path, because one earlier version crashed when RDP was called on `None`.

In the controller, I increased the position and velocity gains from the earlier Project 1 values. The final translational gains I used were

$$K_p = [11, 11, 12], \quad K_d = [7, 7, 5].$$

I also increased the attitude gains to $K_R = [2300, 2300, 100]$ and $K_\omega = [85, 85, 20]$. This helped reduce tracking error without making the quadrotor too unstable. The controller tuning was especially important for the last switchback case, where the path was already collision-free in planning but the vehicle would drift wide during the sharp turns.

3 Extra Credit Implementation

The extra-credit version uses the same controller and VIO estimator, but changes `world_traj.py` and `graph_search.py`. The EC starter code provides an occupancy map that is initially empty and can only be updated within a finite local range around the vehicle. I used that map as the planning map for local replans. Each time the simulator calls `replan`, the planner updates the local map at the current state and searches from the current position to a short local target.

The local target is selected using a global guide path. I still compute a coarse guide path at initialization, but the vehicle does not simply execute that full path. Instead, the guide is used to choose a point a few meters ahead along the route. This avoids the simplest failure mode of local planning, where the local planner repeatedly points straight at the final goal even though the safe corridor bends around obstacles. To avoid getting confused on folded paths

like switchback, I stored a monotonic progress variable along the guide path. This prevents the planner from jumping backward to a nearby but already-passed part of the guide.

I used different local parameters for different route types. Normal maps use a local margin of 0.35 m, local step length of about 3.0 m, and speed of 1.45 m/s. Medium or vertical routes, such as stairwell-style routes, use a larger margin of 0.42 m and speed of 1.55 m/s. Long folded routes, especially switchback, use a separate long-route mode with margin 0.40 m, step length 3.6 m, and speed 1.55 m/s. This was necessary because the switchback map needed enough speed to finish, but also enough replanning discipline to avoid chasing unstable long segments.

I also modified `graph_search.py` so it can search either a full map or the provided local occupancy map. The search uses a 26-connected A* expansion. For local goals, I relaxed the exact goal requirement by allowing the search to snap to a nearby free voxel when the chosen local goal lies inside an obstacle or an unknown blocked region. This was important because the project handout notes that the straight-line local goal can land in an obstacle.

Each EC replan is treated as a new short trajectory starting at the current simulation time. I store the current time as `t0`, and `update` evaluates the command using $t - t0$. This prevents a newly generated local segment from being evaluated as if it had started at the beginning of the flight. If all local A* attempts fail, the planner commands a conservative one-meter fallback segment toward the global goal, or hovers if it is already close enough to the goal. The local trajectory is built as an RDP-style straight-line segment trajectory through shortened waypoints rather than as a long global polynomial, which made frequent replanning easier to track.

4 Why These Changes Were Needed

The biggest lesson was that the shortest geometric path was not always the best flight path. At first I tried smaller margins and higher speeds to reduce time, but that caused collisions in hidden maps. The collision usually came from one of two reasons. First, the smooth polynomial trajectory could cut a corner even if the waypoint path was collision-free. Second, the VIO estimate and controller tracking were not perfect, so the real vehicle needed extra clearance.

The RDP trajectory helped because it simplified the path while still following the A* route. This worked especially well on the window and switchback maps. The main reason I kept both methods was that the reduced-waypoint version alone was not enough, but RDP alone was also not the full answer I wanted. Combining them gave me a better balance between shape, clearance, and speed. For the switchback map, increasing speed first caused collision, so I then tuned the controller gains. After that, the faster RDP trajectory could be tracked well enough to pass with full time points.

The extra-credit planner added a different problem: the vehicle was no longer allowed to plan using the fully known obstacle map at every step. A naive local planner that always aims directly at the final goal can work on simple maps, but it fails in maps with windows, stairwells, and switchbacks because the correct short-term direction can temporarily point away from the final goal. The guide-path local target solved most of this. The remaining challenge was tuning the local step size and speed. If the local step was too small, the vehicle became slow and sometimes timed out. If the step was too long or the speed too high, the quadrotor could drift far from the commanded segment and trigger the fly-away condition.

5 Results

Table 1 shows the final autograder results that I used for this report. All flight times are in seconds. The system passed all six maps without collision and received full credit on each map.

Table 1: Final project performance from the Gradescope autograder.

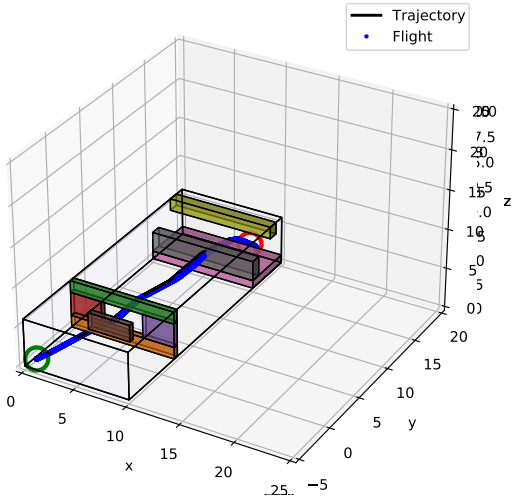
Map	Flight time (s)	Completion	Score
Maze	7.13	pass	16/16
Over-under	9.59	pass	16/16
Window	7.91	pass	16/16
Slalom	19.06	pass	16/16
Stairwell	10.77	pass	18/18
Switchback	19.78	pass	18/18
Total	—	—	100/100

Table 2 shows the final extra-credit performance. Compared with the vanilla global planner in Table 1, the EC version is slower because it replans repeatedly and only uses a local map. For example, maze increased from 7.13 s to 15.58 s, stairwell increased from 10.77 s to 22.60 s, and switchback increased from 19.78 s to 48.42 s. This is the expected

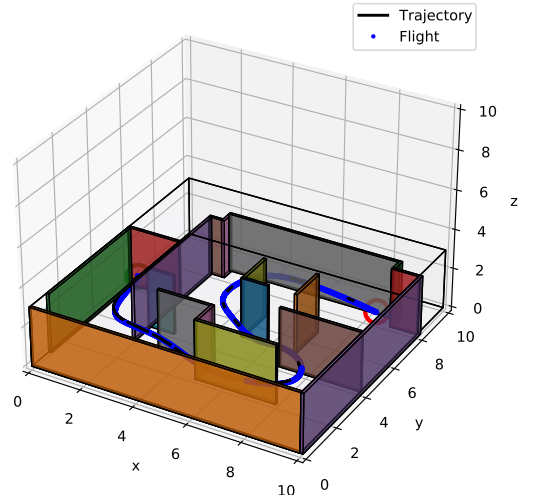
tradeoff for receding-horizon local planning: the system has less map information and must spend extra distance and time correcting the local plan. Even with this added cost, all EC flight times stayed within the relaxed EC time limits and received full extra-credit completion and time points.

Table 2: Final extra-credit local replanning performance.

Map	Flight time (s)	Completion	Score
Maze	15.58	pass	5/5
Over-under	21.20	pass	5/5
Window	16.65	pass	5/5
Slalom	36.95	pass	5/5
Stairwell	22.60	pass	5/5
Switchback	48.42	pass	5/5
Total	—	—	30/30



(a) Window map flight path. Axes are position in m.



(b) Maze map flight path. Axes are position in m.

Figure 1: Example planned trajectory and actual flight results from the local simulator.

6 Discussion

The final planner is not a single perfect method. It is more like a set of practical choices for different map shapes. Smooth polynomial trajectories were good for maps where the path had enough clearance and the motion stayed easy to track. RDP trajectories were better for maps where preserving the exact path geometry mattered more. I also found that tuning only for one map could easily hurt another map, so I tried to make each change local to one class of path when possible.

The hardest main-task case was the aggressive switchback flight. When I first increased the long-map RDP speed, the time improved but the quadrotor collided. This showed that the limiting factor was not just path length, but also tracking accuracy and clearance. The final PID-style tuning made the vehicle more damped and responsive in the sharp turns, so I could use a faster switchback speed of about 3.85 m/s and still avoid collision. A more advanced solution would probably use time allocation based on curvature and obstacle distance instead of one constant speed for the whole route.

For the extra credit, the hardest failure cases were window, over-under, and switchback. In window and over-under, making the local planner too conservative actually made performance worse because the vehicle started taking longer paths and accumulated more tracking error. In switchback, the opposite problem happened first: a fast local planner could reach the goal quickly, but collided or triggered fly-away because the folded guide path caused large tracking errors. The final solution separates normal, medium, and long-route behavior. This is still heuristic, but it gave a useful lesson: local planning is not only about finding a path, but also about choosing local goals that the controller and estimator can safely track.

The EC report requirements ask for implementation details, performance compared to the vanilla simulator, failure cases, and bibliography. The implementation is described in the EC section above: local occupancy-map updates, short-horizon replanning, guide-path local goals, relaxed local-goal search, and separate parameter choices for normal, medium, and long folded routes. The performance comparison is shown in Tables 1 and 2. The main failure cases were also informative: over-conservative local goals made window and over-under unstable, while overly aggressive long-route settings made switchback collide or trigger fly-away. The final version fixed those by keeping normal maps closer to the stable local planner and reserving the special long-route logic for switchback-style maps.

7 Conclusion

Overall, the final system successfully combined A* planning, waypoint simplification, SE(3) control, and VIO state estimation. The most important improvement was making the trajectory less blindly aggressive, choosing the trajectory type based on the map, and then tuning the controller enough to track the difficult switchback case. This gave safe flights on all six main-task maps and a final score of 100/100. The extra-credit version extended the same stack to local replanning with limited sensor range and received 30/30. The project made it clear that planning, control, and estimation cannot be tuned separately when the vehicle only has imperfect state estimates and limited local map information.